

```
• (venv) PS C:\Users\eifnu\NeuroSymbolicEngine> python main.py
--- Nöro-Sembolik Motor Başlatılıyor ---
Hedef kod analiz ediliyor ve Z3 uzayına çevriliyor...

[REDDEDİLDİ] Matematiksel ispat başarısız. Uç durumlar tespit edildi:
- HATA: Sıfıra Bölünme Hatası (Division by Zero)
  AÇIĞI TETİKLEYEN GİRDİ (Counter-example): {'offset': 5, 'divisor': 0, 'y': 5}
  Kritik Açık: Girdiler {'offset': 5, 'divisor': 0, 'y': 5} olduğunda Sıfıra Bölünme Hatası (Division by Zero) meydana geliyor.
• (venv) PS C:\Users\eifnu\NeuroSymbolicEngine>
```

Nöro-Sembolik motor, hedef kodu çalıştırılmadan önce analiz edip Z3 uzayına çeviriyor. Görselde motorun matematiksel ispatı tamamlayamadığı ve kodda ölümcül bir "Sıfıra Bölünme Hatası (Division by Zero)" yakaladığı görülüyor. Daha da önemlisi, sistem sadece hatayı söylemekle kalmıyor; {'offset': 5, 'divisor': 0, 'y': 5} şeklinde bu hatayı tetikleyecek kesin bir matematiksel "Karşı-Örnek" (Counter-example) sunuyor.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(venv) PS C:\Users\eifnu\NeuroSymbolicEngine> python main.py
[İTERASYON 1] Z3 Motoru kodu analiz ediyor...
❌ [REDDEDİLDİ] 1 adet uç durum bulundu.
  -> Sıfıra Bölünme Hatası (Division by Zero) | Girdi: {'offset': 5, 'divisor': 0, 'y': 5}

[BİLGİ] Açıklar Gemini LLM modeline gönderiliyor... Yeni kod sentezleniyor...

[İTERASYON 2] Z3 Motoru kodu analiz ediyor...

✅ [BAŞARILI] Kod %100 güvenli olarak doğrulandı!
-----
def calculate_metrics(x, y):
    offset = 5
    divisor = y - offset
    if divisor == 0:
        raise ZeroDivisionError("Cannot calculate metrics: divisor cannot be zero.")
    result = x / divisor
    return result
-----
(venv) PS C:\Users\eifnu\NeuroSymbolicEngine> █
```

Bu görsel, projenin kalbini oluşturan hibrit döngüyü ispatlıyor.

- [İTERASYON 1]: Z3 motoru kodu analiz ediyor ve sıfıra bölünme hatasını bulup kodu reddediyor.
- [BİLGİ]: Sistem bu hatayı ve parametreleri otonom olarak Gemini LLM modeline gönderiyor.
- [İTERASYON 2]: Gemini'nin sentezlediği yeni kod (içine if divisor == 0: güvenlik duvarı eklenmiş hali) tekrar Z3 motoruna sokuluyor. Sistem bu kez kodu %100 güvenli olarak doğrulayıp onaylıyor.

Neuro-Symbolic Engine API 1.0.0 OAS 3.1

/openapi.json

Matematiksel ispat ve LLM sentezi ile otomatik kod doğrulama motoru.

default

POST

/verify_and_synthesize Verify And Synthesize Code

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "source_code": "def divide_numbers(a, b):\n    return a / b",
  "max_iterations": 3
}
```

Bu görsel, sistemin modüler bir arka uç (backend) servisi olarak dış dünyaya nasıl açıldığını gösterir. Arayüzde, kullanıcının sisteme JSON formatında tehlikeli bir kod bloğu (örneğin; def divide_numbers(a, b): return a / b) ve maksimum iterasyon sınırı gönderdiği istek (request) gövdesi yer alır. Bu durum, motorun farklı uygulamalara (web, mobil veya diğer mikroservisler) kolayca entegre edilebilir bir API formatında çalıştığını kanıtlar.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/verify_and_synthesize' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "source_code": "def divide_numbers(a, b):\n    return a / b",
    "max_iterations": 3
  }'
```

Request URL

```
http://127.0.0.1:8000/verify_and_synthesize
```

Server response

Code	Details
200	Response body <pre>{ "status": "success", "original_code": "def divide_numbers(a, b):\n return a / b", "synthesized_code": "def divide_numbers(a, b):\n if b == 0:\n raise ValueError(\"Cannot divide by zero.\")\n return a / b", "is_safe": true, "iterations_run": 2, "message": "Kod %100 güvenli olarak doğrulandı." }</pre> Download Response headers <pre>content-length: 302 content-type: application/json date: Fri, 29 May 2026 00:16:38 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Bu görsel, Nöro-Sembolik döngünün arka planda işini başarıyla bitirdiğinin kesin kanıtıdır. Sunucu saniyeler içinde kodu analiz etmiş, Z3 ile açığı bulmuş ve JSON yanıtı içerisinde orijinal koda dokunmadan yeni bir "synthesized_code" (sentezlenmiş kod) bloğu döndürmüştür. Üretilen kodun içinde yapay zekanın eklediği if b == 0: raise ValueError... güvenlik duvarı açıkça görülmektedir. Ayrıca yanıt gövdesindeki "is_safe": true ve "Kod %100 güvenli olarak doğrulandı." mesajları, üretilen kodun sisteme verilmeden önce matematksel olarak tekrar doğrulandığını (Çift Dikiş / Closed-Loop) belgeler.

```
● (venv) PS C:\Users\eifnu\NeuroSymbolicEngine> python -m src.ci_cd.github_pr_bot  
🚗 [BOT BAŞLATILDI] Hedef Repo: ElifNur06/NeuroSymbolicEngine | Dosya: main.py  
🔍 Nöro-Sembolik analiz yapılıyor...  
✅ Kod %100 güvenli. Herhangi bir PR açılmasına gerek yok.
```

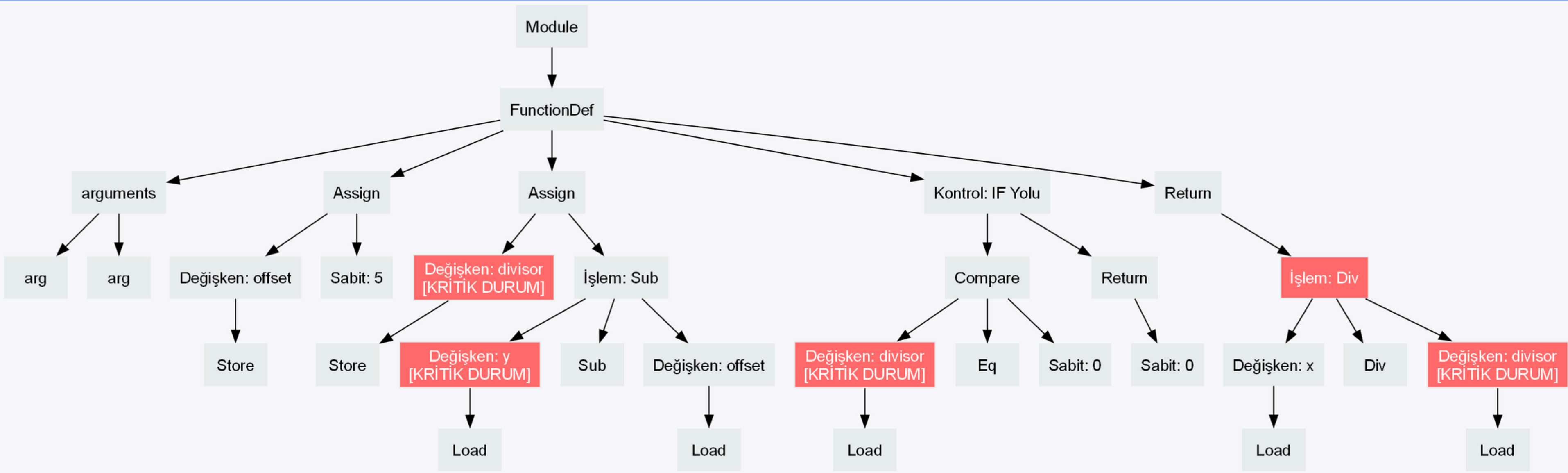
Bu görsel, sistemin DevOps ve CI/CD süreçlerindeki pasif denetim (monitoring) yeteneğini sergiler. Bot, hedef repodaki main.py dosyasını tarayarak Nöro-Sembolik analiz sürecini başlatır. İnceleme sonucunda herhangi bir mantıksal zafiyet veya uç durum (edge case) bulunmadığı için motor "Kod %100 güvenli" onayını verir. Bot, kodun güvenli olduğunu anladığı için sisteme müdahale etmez ve gereksiz bir PR açmayarak geliştirme sürecinin doğal akışında devam etmesine izin verir.

```
• (venv) PS C:\Users\eifnu\NeuroSymbolicEngine> python -m src.ci_cd.github_pr_bot
🚗 [BOT BAŞLATILDI] Hedef Repo: ElifNur06/NeuroSymbolicEngine | Dosya: test_hata.py
🔍 Nöro-Sembolik analiz yapılıyor...
⚠️ Açık Bulundu! 1 adet uç durum (edge case) tespit edildi.
🧠 Gemini (LLM) üzerinden güvenli kod sentezleniyor...
✅ Sentezlenen kod güvenlik onayını aldı. GitHub'a yükleniyor (Push & PR)...
🎉 BAŞARILI! Pull Request oluşturuldu: https://github.com/ElifNur06/NeuroSymbolicEngine/pull/1
```

Bu görsel, CI/CD entegrasyonunun en can alıcı noktası olan otonom inisiyatif alma gücünü belgeliyor. Bot bu kez içinde kasıtlı olarak bir uç durum (edge case) barındıran test_hata.py dosyasını tarar. Matematiksel açığı tespit ettiği anda inşayı durdurur, Gemini LLM üzerinden kodun güvenli halini sentezler ve geliştiriciye yük olmadan doğrudan GitHub reposuna bir Pull Request (PR) açar. Terminal ekranında beliren <https://github.com/ElifNur06/NeuroSymbolicEngine/pull/1> linki, bu proaktif güvenlik yamasının başarıyla GitHub'a itildiğini (push) kesin olarak kanıtlamaktadır.

```
• (venv) PS C:\Users\eifnu\NeuroSymbolicEngine> python -m src.core.symbolic.state_verifier
=== Nöro-Sembolik Asenkron Mimari Doğrulaması Başlıyor ===
✗ MİMARİ ÇÖKÜŞ TESPİT EDİLDİ!
  -> Hata: Asenkron Durum Yönetimi Çökmesi (State Update After Dispose)
  -> Çözüm Önerisi: Kritik Açık: API isteği arka planda çözümlendiğinde, UI çoktan kapatılmış (disposed) olabilir. UI güncellemesi yapılmadan önce 'isMounted' veya aktiflik kontrolü yapılmalıdır.
• (venv) PS C:\Users\eifnu\NeuroSymbolicEngine> python -m src.core.symbolic.visualizer
[BİLGİ] Çizim motoru bulundu ve eklendi: C:\Program Files\Graphviz\bin
✓ Görsel İspat Haritası oluşturuldu: ast_logic_graph.png
```

Sistemin mobil ve ön yüz mimarilerindeki karmaşık sorunları yakalama yeteneği sergileniyor. Motor, kod çalışmadan "Asenkron Durum Yönetimi Çökmesi" (State Update After Dispose) hatasını tespit ediyor. Arka planda bir API yanıtı beklenirken kullanıcının ekranı kapatması (dispose) durumunda uygulamanın çökeceğini kanıtlıyor ve isMounted kontrolü eklenmesi gerektiğini öneriyor.



src.core.symbolic.visualizer modülü tarafından üretilen ast_logic_graph.png dosyası , kodun Soyut Sözdizimi Ağacı (AST) ve Z3 zafiyet haritası. Bu görsel, projedeki yapay zekanın "Kara Kutu" (Black Box) hissiyatını yıkan en güçlü şeffaflık kanıtıdır. Motor, kodu sadece düz bir metin olarak okumakla kalmıyor; onu en temel mantıksal yapı taşlarına (Module, FunctionDef, Assign, Return) ayırarak uzaysal bir ağaç çiziyor. Haritadaki kırmızı renkli ve [KRİTİK DURUM] etiketli düğümler, Z3 motorunun bulduğu hatanın kök nedenini (root cause) adım adım izliyor. Sorunun kaynağı olan y değişkeninden başlayıp, tehlikeli divisor hesaplamasına geçiyor ve nihayetinde uygulamanın tam olarak çökeceği o ölümcül İşlem: Div (bölme işlemi) anını kabak gibi ortaya çıkarıyor.

```
• (venv) PS C:\Users\eifnu\NeuroSymbolicEngine> python -m src.core.symbolic.smart_contract_verifier

[WEB3 ANALİZ] Akıllı Kontrat Para Çekme Mantığı İnceleniyor...
✗ AKILLI KONTRAT ZAFİYETİ TESPİT EDİLDİ!
  -> Hata Tipi: Bakiye Üstü Çekim / Tamsayı Taşması (Integer Underflow)
  -> Hack Senaryosu: {'kullanici_bakiyesi': 0, 'cekilecek_miktar': 1}
  -> Çözüm: Kritik Açık: Kullanıcı hesabında 0 token varken, 1 token çekebiliyor. 'require(balance >= amount)' güvenlik kontrolü eksik!
○ (venv) PS C:\Users\eifnu\NeuroSymbolicEngine> █
```

Sistemin blokzincir ekosistemine entegrasyonu. Motor, bir para çekme (withdraw) fonksiyonunu analiz ediyor ve ölümcül bir "Bakiye Üstü Çekim / Tamsayı Taşması (Integer Underflow)" finansal zafiyeti tespit ediyor. Hesabında 0 token olan bir hacker'ın 1 token çekerek akıllı kontratı çökertebileceği matematiksel olarak kanıtlanıyor.